

CS779 Competition: Machine Translation System for India

Om Mallick
251110055
ommallick25@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

In this report, we present an approach for Neural Machine Translation (NMT) of English text into Bengali and Hindi. For this purpose, we experimented with different architectures involving Seq2Seq GRU, LSTM, or Transformer-based models, pre-trained static word embeddings such as FastText, and others. The models were implemented from scratch using PyTorch. The best-performing model was a **Bidirectional GRU Encoder–Decoder with Cross Attention and Beam Search Decoding** which achieved a **chrF++ score of 0.331**, a **ROUGE score of 0.431**, and a **BLEU score of 0.123**. It ranked **58th** on the test phase leaderboard.

1 Competition Result

Codalab Username: o_251110055

Final Leaderboard Rank on the Test Set: 58

chrF++ Score corresponding to the Final Rank: 0.331

ROUGE Score corresponding to the Final Rank: 0.431

BLEU Score corresponding to the Final Rank: 0.123

Total Number of Submissions in the Training Phase: 25

Total Number of Submissions in the Testing Phase: 5

Table showing the Number of Submissions made each Week during the Training Phase:

Week #	Dates	# of Submissions	Best Score Till Now (chrF++)
Week 1	Oct 6–12, 2025	2	0.14
Week 2	Oct 13–19, 2025	4	0.14
Week 3	Oct 20–26, 2025	4	0.14
Week 4	Oct 27–Nov 2, 2025	5	0.27
Week 5	Nov 3, 2025	10	0.34

2 Problem Description

The aim of this project was to develop an NMT system that can translate English text into Bengali and Hindi, two important Indian languages. This was motivated by the emerging requirement of multilingual access in digital media and news publishing where a huge amount of English content needs to be translated effectively for local readers.

Unlike the usual rule-based or phrase-based models, in this work, the focus was on deep learning-based neural models that learn translation patterns from bilingual corpora directly. The system employed an encoder–decoder architecture [1]: a neural framework introduced for sequence-to-sequence learning tasks like machine translation.

No pre-trained contextual models like BERT or mBART were allowed; instead, all models were implemented from scratch using PyTorch. Only static embeddings such as FastText or GloVe were allowed. Various architectures were systematically evaluated under GPU compute constraints to find the best configuration: Transformer-based encoder–decoders and recurrent networks with LSTM or GRU cells.

This work aimed at testing the understanding of statistical NLP, neural architectures, and optimization under resource limitations while producing a deployable translation model capable of practical utility for low-resource Indian languages.

3 Data Analysis

The competition dataset included parallel corpora for the English–Bengali and English–Hindi language pairs stored in JSON format. Each corpus was structured to have a train/test split. The English–Bengali dataset had around 68,849 training and 19,672 test entries, and the English–Hindi dataset had around 80,797 training and 23,085 test entries, and each entry included one English source sentence and its translation.

For the English–Bengali corpus, the vocabulary sizes were as follows: 65,381 (English) and 92,727 (Bengali). For the English–Hindi corpus, the vocabulary sizes were as follows: 70,172 (English) and 69,499 (Hindi). This discrepancy reflects the morphological richness and script diversity of Indian languages, which can have compound words and inflectional variations.

Statistical analysis also showed that Hindi and Bengali sentences averaged $1.2\times$ and $1.4\times$ more characters respectively than English sentences, confirming longer average lengths. We visualized the frequency of tokens (Figure 1) where we noted a strongly Zipfian trend with a long tail of rare words, which is one of the main factors in deciding on the presence of an <UNK> token during encoding.

The test data had similar structural properties to the data in the training corpus, but had unseen named entities, and thus exhibited some mild domain shifts. For this reason, we employed a beam search decoding strategy to ensure robustness in these situations.

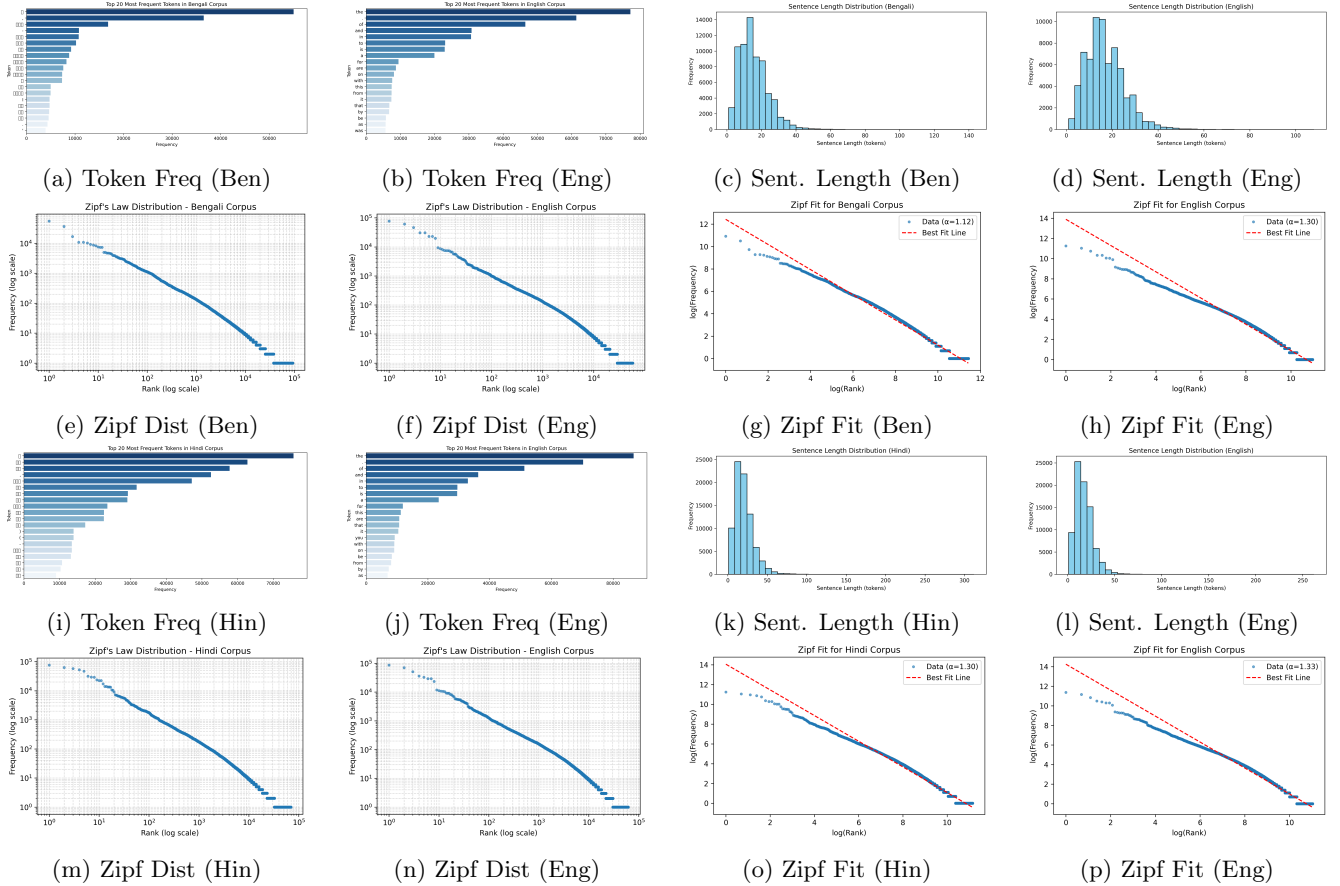


Figure 1: Corpus statistics.

4 Model Description

In this section, we describe the architectural transformation of the system during the competition, and we document experiments, architectural changes, and reasons that led to the final model design.

4.1 Model Evolution and Experimental Pathway

The model used as the baseline model from the starter kit was a straightforward and simple unidirectional GRU-based encoder-decoder architecture without attention. From this point onward, multiple changes to the architecture were made, including switching between GRU and LSTM cells, unidirectional and bidirectional, and a variety of attention variants.

We continued experimenting with multiple configurations after the last one did not work, and some of the modified configurations met implementation difficulties, e.g., mismatches in the tensor dimensions, others did not perform well due to excessive overfitting or high GPU memory usage. We tried the following major variants:

- **Bidirectional Encoders:** allowed for context flow from both directions, which improved how well the source was represented.
- **Pretrained embeddings (FastText):** limited improvements at a significant increase in training time and memory availability.
- **Transformer Encoder-Decoder:** sometimes improved performance but significantly increased computational load, and didn't work without BPE tokenization [2].

4.2 Final Model Architecture

After many iterations of experimentation, we settled on a final optimized architecture consisting of a **Bidirectional GRU Encoder-Decoder with Cross Attention**. Figure 2 presents the overall schematic.

- **Encoder:** Each token in the source text is embedded into a continuous vector space and processed through a bidirectional GRU. The forward and backward hidden states are concatenated and linearly projected to produce the shared context.
- **Decoder:** A single-layer GRU produces target tokens one at a time and uses additive attention to produce a context vector from the encoder output.
- **Attention Mechanism:** Following Bahdanau et al. (2015)[3], an attention score is computed as a learned score of the encoder hidden state and decoder hidden state to enable the decoder to attend to relevant source words.
- **Regularization:** Dropout layers were added to the embedding and GRU outputs to reduce overfitting.

4.3 Loss Function and Optimization

The model was trained using **Cross-Entropy Loss** with the <PAD> tokens skipped to avoid crossing the gradient by calculating the loss. The optimizer was **Adam** with weight decay regularization. A custom **Warmup-Cosine Learning Rate Scheduler** was used to stabilize convergence. For a sequence of target tokens $y_1, y_2, \dots, y_T$, the loss is given by:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log P(y_t | y_{<t}, x)$$

4.4 Inference and Decoding Strategy

Decoding began with a **Greedy Search** approach, which was later replaced with **Beam Search**, tracking the top- k translation hypotheses at each step. Beam search yielded grammatically coherent translations for longer sequences. Other strategies such as **Top- k Sampling** and **Speculative Decoding** were evaluated but produced inferior results; hence, beam search was adopted for final inference.

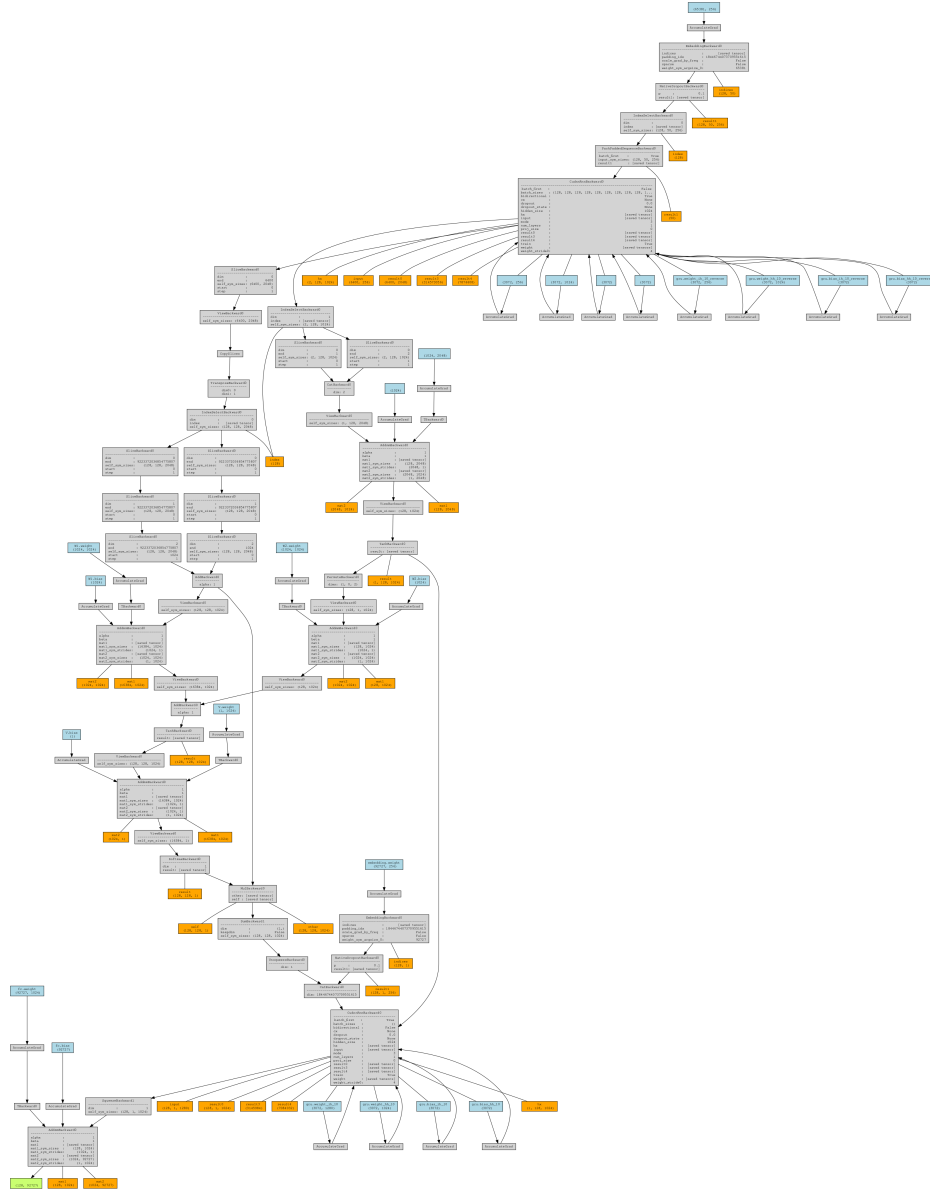


Figure 2: Final model architecture: Bidirectional GRU Encoder-Decoder with Cross Attention.

4.5 Key Learnings

- The attention mechanism played a large role in improving translation quality for longer and more morphologically rich sentences.
- LSTM variants did not produce a significant improvement over GRU while simultaneously consuming more processing power.
- Pretrained embeddings did not produce significant gains and were computationally expensive relative to other approaches.
- Beam search decoding did produce the most impactful improvement in translation fluency.

5 Experiments

This section summarizes our experimental procedure, including preprocessing of data, training configurations, and hyperparameter exploration.

5.1 Data Preprocessing

Preprocessing was a vital element affecting the performance of the model. Every preprocessing step was performed to account for cleanliness, token-level parallelism, and to ensure harmony between source and target corpora. The key operations included the following.

- **Normalization:** We ensured Unicode normalization and removal of special symbols on the source text using the IndicNLP Library’s normalizers that ensured uniform treating of diacritics.
- **Tokenization:** We used NLTK’s `word_tokenize()` method to perform tokenization for the English sentences and an IndicNLP trivial tokenizer for subword tokenization of Indic languages.
- **Noise Removal:** We removed stray punctuation, extra spaces and unintended words from the English sentences (in Hindi/Bengali sentences).
- **Padding and Limiting Vocabulary:** For the sequences we fixed all the sequences to be to be 50 tokens with the addition of special tokens (`<s>`, `</s>`, `<pad>`) to mark the beginning, padding, and end of a sentence.

The processed data was converted into tensors, leveraging the PyTorch library’s `DataLoader`, which enabled faster convergence and better stability while training.

5.2 Training Procedure

All experiments were carried out on a single NVIDIA Tesla P100 GPU within a Kaggle environment. Each epoch took an average of 30–40 minutes of training time, depending on model complexity. The **Adam** optimizer was used with a learning rate of 1×10^{-3} and a weight decay term of 1×10^{-5} . We used a **Warmup-Cosine scheduler** with 10% warmup in the optimization process. We also used gradient clipping (L_2 norm of 1.0) to effectively reduce the issues associated with gradient explosions.

- **Loss Function:** Cross-Entropy Loss (ignoring padding).
- **Batch Size:** 256.
- **Epochs:** 10.
- **Teacher Forcing Ratio:** 0.5.

5.3 Hyperparameter Exploration and Model Variants

As indicated in Table 1, we have attempted multiple configurations during the hyperparameter tuning process. For morphologically complex languages, we realised that increasing hidden units from 512 to 1024 increases performance. Additionally, dropout values greater than 0.3 led to underfitting; 0.1 was optimal.

Model Variant	Hid. Units	Embd. Dim	Dropout	LR
GRU (Unidirectional, No Attention)	512	128	0.2	0.001
GRU (Bidirectional, Attention)	512	128	0.2	0.001
LSTM (Unidirectional, No Attention)	512	128	0.2	0.001
LSTM (Bidirectional, Attention)	512	128	0.2	0.001
GRU (Bidirectional, Attention)	1024	256	0.1	0.001
GRU (Bidirectional, Attention) + FastText	1024	300	0.1	0.0007
Transformer w/o BPE (4 Layers, 8 Heads)	512	256	0.2	0.0005
Transformer w BPE (4 Layers, 8 Heads)	512	256	0.2	0.0005
GRU (Bidirectional, Attention) + Top K	1024	256	0.1	0.001
GRU (Bidirectional, Attention) + Spec Decoding	1024	256	0.1	0.001
GRU (Bidirectional, Attention) + Beam (Final)	1024	256	0.1	0.001

Table 1: Summary of key hyperparameter configurations across different model variants.

5.4 Training Observations

The bidirectional GRU models with cross attention converged faster, and exhibited less uneven loss decay. The transformer models used approximately three times as much GPU memory while achieving only minor improvements in accuracy. The final GRU model achieved the best tradeoff between accuracy and overall resources used.

6 Results

This section presents the quantitative results of the submissions during both training and testing phases on CodaLab. Each submission represents a separately trained model checkpoint or a variation in decoding strategy, optimizer scheduling, or preprocessing pipeline. All submissions were evaluated using **chrF++**, **ROUGE**, and **BLEU** scores.

6.1 Training Phase Results

During the training phase from October 6 to November 3, 2025, a total of 25 submissions were made. Submissions at the start were primarily aimed at testing our baseline architectures and evaluation of data preprocessing, with later submissions demonstrating systematic refinements over time through the introduction of **bidirectional encoders**, **cross-attention**, and **beam search decoding**.

As illustrated in Table 1, model performance improved steadily through submissions, which demonstrates that we can leverage incremental tuning and architectural modifications. The addition of **attention**, **bidirectionality**, and **beam search decoding** specifically improved fluency and consistency of translation.

6.2 Testing Phase Results

The testing phase had five submissions made over the period of November 5 - November 6, 2025. The best model on the test set achieved a **chrF++ score of 0.331**, **ROUGE score of 0.431**, and **BLEU score of 0.123**.

6.3 Final Results

The final **Bidirectional GRU Encoder–Decoder with Cross Attention and Beam Search Decoding** model outperformed all preceding configurations. It’s performance can be attributed to several factors:

- It used a **bidirectional encoder** in order to capture both forward and backward contextual dependencies, which improved source sentence representation.
- The **cross-attention** mechanism gave dynamic attention to the relevant source tokens, enhancing the translation of long and morphologically complex sentences.
- **Beam search decoding** mitigated locally optimal but globally incoherent translations.
- The **Warmup-Cosine scheduler** with gradient clipping ensured stabilization in the training process, preventing divergence.

Models without **attention**, or using **greedy decoding**, produced truncated or repetitive translations. **Transformer-based architectures** tended to perform worse in this regard, largely due to the limited dataset and GPU memory constraints.

Submission ID	Date	Status	chrF++ Score
388923	Oct 12, 2025	Finished	0.12
389906	Oct 13, 2025	Finished	0.14
395486	Oct 19, 2025	Finished	0.09
395747	Oct 19, 2025	Finished	0.10
401439	Oct 26, 2025	Finished	0.12
409359	Oct 26, 2025	Finished	0.18
409543	Nov 02, 2025	Finished	0.26
409628	Nov 02, 2025	Finished	0.16
409702	Nov 02, 2025	Finished	0.18
409798	Nov 02, 2025	Finished	0.27
410080	Nov 03, 2025	Finished	0.15
410725	Nov 03, 2025	Finished	0.32
410746	Nov 03, 2025	Finished	0.26
412121	Nov 03, 2025	Finished	0.28
412149	Nov 03, 2025	Finished	0.29
412185	Nov 03, 2025	Finished	0.32
412424	Nov 03, 2025	Finished	0.34
412428	Nov 03, 2025	Finished	0.27
412555	Nov 03, 2025	Finished	0.34 (Best Score)

Table 2: Training phase submissions and corresponding chrF++ scores.

Submission ID	Date	Status	chrF++ Score
414099	Nov 05, 2025	Finished	0.33 (Final Submission)
414327	Nov 05, 2025	Finished	0.15
414395	Nov 05, 2025	Finished	0.10
414689	Nov 05, 2025	Finished	0.28
414747	Nov 06, 2025	Finished	0.31

Table 3: Testing phase submissions and corresponding chrF++ scores.

7 Error Analysis

This section gives the qualitative analysis of the translation errors, limitations of the developed architectures, and insights into the pattern of translation inaccuracies across both the English–Hindi and English–Bengali tasks.

7.1 Comparative Model Behavior

The experimental analysis showed that the unidirectional GRU and LSTM models without attention often failed to preserve the sentence structure, frequently omitting tokens or repeating others. These models performed decently when working with simple short sentences, but their output was quite poor when dealing with longer sentences or compound ones including subordinate clauses.

The Bidirectional GRU with Cross Attention produced translations that were more coherent and contextually appropriate in both target languages. However, it occasionally did produce semantic errors by literally translating idiomatic or context dependent expressions.

7.2 Common Error Types

A structured analysis of model predictions identified three significant categories of errors:

1. **Inversion of Word Order:** Owing to differences in structure (SVO vs SOV in English vs Hindi/Bengali), the model occasionally misplaced objects or auxiliaries, especially in complex sentences.

2. **Omission of Function Words:** Articles and conjunctions such as "and", "the", and "of" were sometimes omitted, probably due to low relative frequency in the target corpus or weak attention focus.
3. **Named Entities:** Unseen proper nouns or regional names were either transliterated inconsistently or replaced by slightly different tokens for lack of exposure to proper nouns during training.

7.3 Reasons for Imperfection and Possible Improvements

While this model generalized very well, several factors limited its ultimate accuracy:

- **Limited parallel data:** The available bilingual corpora did not sufficiently cover domain-specific terms or the full morphological richness of Hindi and Bengali.
- **Static Embeddings:** The model, without having contextual embeddings such as **mBERT** or **XLNet**, had difficulty in disambiguating polysemous words according to their contexts.
- **Attention Diffusion:** Long or syntactically complex sentences create less focused attention weights, resulting in phrase-level distortions.
- **Absence of Subword Segmentation:** The lack of **BPE** or **SentencePiece** tokenization of the Indic scripts resulted in vocabulary sparsity and degraded rare-word embeddings.

7.4 Interesting Insights

Despite these difficulties, some striking observations were made:

- **Beam search decoding** greatly improved the completeness of translations, while greedy decoding often prematurely predicted the end-of-sequence token **<EOS>**.
- Manual evaluation confirmed that the model sometimes generated sentences which were fluent but semantically wrong—an indication of strong target-side fluency learned without explicit reinforcement.

8 Conclusion

This work presented an end-to-end NMT system for English–Hindi and English–Bengali translations under constrained resources. The study, via systematic experimentation, established that recurrent models—when optimized with attention mechanisms and decoding strategies—can outperform more complex transformer models in low-resource settings.

Key insights:

- Attention mechanisms improve the token-level alignment and translation qualities.
- Well-tuned recurrent models are both computationally efficient and generalize effectively.
- Beam search decoding improves fluency and completeness.
- Indic language preprocessing significantly affects convergence stability.

Future Work:

- Incorporating multilingual contextual embeddings, either **mBERT** or **IndicBERT**.
- Employing subword tokenization techniques like **BPE** or **SentencePiece**.
- Exploring hybrid transformer–recurrent architectures.

References

- [1] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in *Advances in neural information processing systems*, pp. 3104–3112, 2014.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in neural information processing systems*, pp. 5998–6008, 2017.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” *arXiv preprint arXiv:1409.0473*, 2014.